



NextGen

Efficient Python Lecture 04

CERN
STEAM Academy

10 July 2026

Idiomatic Scientific Python

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland



Outline

Numerical types, precision, and floating-point control

State, ownership, and hidden coupling

Function interfaces and explicit data flow

Modules, scripts, and lazy data flow

Maintainability tools for scientific code

Further reading

Section 1

Numerical types, precision, and floating-point control

Why types matter in scientific Python

In Python, we usually do not declare variable types in the way we would in C or C++. Instead, values are runtime objects, and arrays carry their own numerical representation.

```
1 import numpy as np
2
3 n = 3
4 x = 3.0
5 z = 1.0 + 2.0j
6
7 a = np.array([1, 2, 3])
8 u = np.array([1.0, 2.0, 3.0], dtype=np.float64)
9
10 print(type(n), type(x), type(z))
11 print(a.dtype, u.dtype)
```

There are two levels to keep separate:

- ▶ the Python object type, such as `int`, `float`, `complex`,
- ▶ the NumPy array dtype, such as `int64`, `float32`, `float64`.

For numerical work, the dtype of an array is often more important than the type of the Python name that refers to it.

Python numbers and NumPy dtypes are not the same

Python integers have arbitrary precision. NumPy integer arrays usually have fixed-width storage, closer to what we know from C and C++.

```
1 py_value = 30_000 + 30_000
2 print(py_value)           # 60000
3
4 a = np.array([30_000], dtype=np.int16)
5 print(a + a)              # may overflow: [-5536]
6 print((a + a).dtype)      # int16
```

Fixed-width types are useful: they make memory usage predictable and allow efficient array operations. But they also mean that overflow is possible.

This matters when reading binary data, interfacing with C/C++ libraries, storing detector counts, using integer labels, or reducing memory usage deliberately.

A good habit is to choose integer dtypes deliberately whenever the range of values matters.

Floating-point arithmetic is finite arithmetic

A floating-point number is not a real number. It belongs to a finite set of representable values. Most real numbers are rounded when stored.

```
1 print(0.1 + 0.2 == 0.3)
2
3 info = np.finfo(np.float64)
4
5 print(info.eps)      # distance from 1.0 to the next larger float64
6 print(info.tiny)     # smallest positive normal float64
7 print(info.max)      # largest finite float64
8
9 x = 1.0
10 print(np.nextafter(x, 2.0) - x)
```

The problem is not that floating-point arithmetic is bad. It is that it is approximate, rounded arithmetic with a well-defined precision range.

In scientific computing, numerical precision is part of the experimental setup, just like grid spacing, time step, solver tolerance, or measurement uncertainty.

Choosing precision deliberately

The dtype controls both precision and memory use. In NumPy, floating-point arrays are often `float64` by default, but `float32` is common in machine learning, GPUs, and large data workflows.

```
1 x64 = np.linspace(0.0, 1.0, 1_000_000, dtype=np.float64)
2 x32 = x64.astype(np.float32)
3
4 print(x64.nbytes / 1024**2)    # memory in MiB
5 print(x32.nbytes / 1024**2)
6
7 s1 = np.sum(x32)                # accumulator chosen by NumPy
8 s2 = np.sum(x32, dtype=np.float64)
9
10 print(s1, s2)
```

Using `float32` may reduce memory and increase throughput. It may also change the accuracy of sums, residuals, gradients, and stopping criteria.

Changing precision should therefore be treated as a numerical modelling decision, not only as a performance trick.

Stable formulas can matter more than more digits

Increasing precision is not the only way to improve numerical reliability. Often, the more important step is to use a stable mathematical form.

```
1 x = 1e-12
2 a = np.log(1.0 + x)
3 b = np.log1p(x)
4 print(a)
5 print(b)
6 y = 1e8
7 bad = np.sqrt(y + 1.0) - np.sqrt(y)
8 good = 1.0 / (np.sqrt(y + 1.0) + np.sqrt(y))
9 print(bad)
10 print(good)
```

The expression may be mathematically correct but numerically fragile. Subtracting nearly equal numbers can destroy significant digits.

Scientific Python provides functions such as `np.log1p` and `np.expm1` precisely because stable formulations are part of numerical practice.

Control floating-point warnings

Invalid values are not only programming errors. They are often numerical evidence: overflow, division by zero, invalid domains, or model assumptions being violated.

```
1 u = np.array([1.0, 0.0, -1.0])
2
3 with np.errstate(divide="raise", invalid="raise"):
4     try:
5         y = np.log(u)
6     except FloatingPointError as err:
7         print("floating-point problem:", err)
8
9 finite = np.isfinite(u)
10 print(finite)
```

During exploration, warnings may be enough. During a trusted computation, it may be better to fail early and inspect the input.

A robust numerical workflow does not silently continue after producing NaN, inf, or invalid intermediate values unless this behaviour is intentional and documented.

Compare floating-point results with tolerances

Exact equality is rarely the right question for floating-point arrays. A better question is whether two results agree within a tolerance justified by the numerical scale of the problem.

```
1 def relative_error(u, u_ref):
2     return np.linalg.norm(u - u_ref) / np.linalg.norm(u_ref)
3
4 ok = np.allclose(
5     u,
6     u_ref,
7     rtol=1e-10,
8     atol=1e-12,
9 )
10
11 err = relative_error(u, u_ref)
```

A tolerance is not a magic number. It should reflect the expected scale of round-off error, discretization error, solver tolerance, measurement noise, or modelling uncertainty. The practical rule is simple: do not only ask whether two arrays are equal. Ask what level of difference would be scientifically meaningful.

Section 2

State, ownership, and hidden coupling

From numerical workflows to reliable programs

In the previous lectures, we treated arrays as computational representations of mathematical objects, numerical experiments, and learning workflows. We now move one level up: from individual computations to programs that can be trusted, modified, tested, and reused.

data $\longrightarrow$ parameters $\longrightarrow$ computation $\longrightarrow$ diagnostics

The central question of this lecture is not whether Python can express a formula. It is how to write scientific Python code so that the formula, the assumptions, the data flow, and the state of the computation remain visible.

This matters before optimization. Code that hides state or mixes unrelated responsibilities is difficult to test, difficult to profile, and dangerous to accelerate.

What idiomatic means in scientific Python

In this lecture, idiomatic Python does not mean writing clever or compact code. It means writing code whose structure matches the scientific workflow.

Good scientific Python code should make it clear:

- ▶ what data enter a computation,
- ▶ what assumptions are made about shapes, units, dtypes, and parameters,
- ▶ what state is modified,
- ▶ what result is returned,
- ▶ where a numerical error or modelling choice can be inspected.

The goal is not style for its own sake. The goal is reliability under change. A script that works once is useful. A program that can survive a new parameter, a new dataset, a new solver, or a new collaborator is much more valuable.

Hidden coupling is the real enemy

A computation is easy to reason about when its result depends on what we can see at the call site. Hidden coupling appears when the result also depends on something implicit.

hidden coupling = visible inputs + invisible dependencies.

Typical sources of hidden coupling in scientific Python are:

- ▶ global parameters used inside functions,
- ▶ shared arrays modified in place,
- ▶ notebook variables left over from earlier cells,
- ▶ random-number generators hidden inside helper functions,
- ▶ cached values, files, or temporary outputs reused accidentally.

The problem is not that state exists. Scientific computations need state. The problem is state that cannot be inspected, reset, or controlled.

Array ownership: input, output, scratch

For arrays, a useful design question is: who owns this data, and who is allowed to modify it?

```
1 def residual_1d(u, dx, out):  
2     out.fill(0.0)  
3  
4     out[1:-1] = (  
5         -(u[:-2] - 2.0*u[1:-1] + u[2:])) / dx**2  
6         + u[1:-1]*(u[1:-1]**2 - 1.0)  
7     )  
8  
9     return out
```

This interface has a clear ownership contract:

- ▶ `u` is an input array and should be read, not modified,
- ▶ `out` is an output array and will be overwritten,
- ▶ temporary work is local to the function.

This style is more explicit than a function that silently changes one of its inputs.

Read-only arrays as a debugging tool

When a reference solution, initial condition, or raw dataset should not be modified, we can make this intention executable.

```
1 u0 = np.tanh(x)
2 u_ref = u0.copy()
3
4 u_ref.setflags(write=False)
5
6 # This will raise a ValueError:
7 # u_ref[0] = 0.0
```

This is not something we need everywhere. But it is very useful when debugging larger workflows.

If a function accidentally writes to a read-only reference array, the program fails at the line where the unintended mutation occurs, rather than much later when a diagnostic looks wrong.

Copy at ownership boundaries

Copies should not be used randomly. They should be used where the ownership of data changes.

```
1 def prepare_state(u0):  
2     # New simulation state: independent from the input object  
3     return np.array(u0, dtype=np.float64, copy=True)  
4  
5 def interior_view(u):  
6     # Read-only analysis window: no ownership transfer  
7     return u[1:-1]  
8  
9 u = prepare_state(u0)  
10 inner = interior_view(u)  
11  
12 print(np.shares_memory(u, inner))
```

The question is not simply “view or copy?”. The better question is: should this object be an independent numerical quantity, or is it only a temporary way of looking at existing data? Copying deliberately at ownership boundaries makes later mutation much safer.

Controlled closures instead of hidden globals

Sometimes a function required by a library must have a specific signature. In that case, a closure can be useful, but it should be constructed deliberately.

```
1 def make_decay_rhs(alpha, forcing):
2     def rhs(t, y):
3         return -alpha*y + forcing(t)
4     return rhs
5
6 def zero_forcing(t):
7     return 0.0
8
9 rhs = make_decay_rhs(alpha=0.2, forcing=zero_forcing)
```

The function `rhs` still remembers `alpha` and `forcing`, but the dependency was created explicitly.

This is different from reading a changing global variable inside the function. A controlled closure is a small object representing a chosen model configuration.

Notebook state should become dependencies

Notebooks are useful for exploration, visualization, and teaching. But a notebook has hidden state: variables can survive from earlier cells, cells can be executed out of order, and old definitions can remain active after the text has changed.

```
1 # Exploratory phase
2 alpha = 0.1
3 dx = 0.01
4
5 def rhs(u):
6     return laplacian(u, dx) - alpha*u
```

This may be acceptable during quick exploration. Once the calculation becomes important, the dependencies should move into the function interface.

```
1 def rhs(u, *, dx, alpha):
2     return laplacian(u, dx) - alpha*u
```

A good rule is simple: restart the kernel and run the notebook from top to bottom. If that fails, the notebook contains hidden dependencies.

Randomness is part of reproducibility

Random numbers are not just numbers. They come from a generator with internal state. For simulations, synthetic data, stochastic optimization, and machine learning, this state should be explicit.

```
1 def add_noise(u, *, sigma, rng):  
2     noise = sigma * rng.normal(size=u.shape)  
3     return u + noise  
4  
5 rng = np.random.default_rng(123)  
6  
7 x = np.linspace(0.0, 1.0, 500)  
8 u = np.sin(2*np.pi*x)  
9  
10 u_noisy = add_noise(u, sigma=0.05, rng=rng)
```

Passing `rng` explicitly makes the source of randomness visible. It also makes the function easier to test, because reproducibility is controlled by the caller, not hidden inside the function.

Default arguments can accidentally store state

Not all hidden state comes from arrays. A classic Python trap is a mutable default argument.

```
1 def record_loss_bad(value, history=[]):
2     history.append(value)
3     return history
4
5 print(record_loss_bad(1.0))
6 print(record_loss_bad(2.0))    # uses the same list again
```

The default list is created once, when the function is defined, not each time the function is called.

A safer pattern creates a new object inside the call:

```
1 def record_loss(value, history=None):
2     if history is None:
3         history = []
4
5     history.append(value)
6     return history
```

This matters in notebooks, training loops, diagnostics, logging helpers, and small utilities that slowly become part of a larger project.

A state policy for scientific code

A useful rule is not “never mutate”. Mutation is sometimes exactly what we want. The rule is: make mutation local, named, and testable.

```
1 def euler_step(u, dt, params):  
2     return u + dt * rhs(u, params)  
3  
4 def euler_step_in_place(u, dt, params, scratch):  
5     scratch[:] = rhs(u, params)  
6     u[:] += dt * scratch  
7     return u
```

The first function returns a new array. The second function intentionally modifies `u`, uses `scratch` as temporary storage, and says so in its name.

This separation is especially helpful when a project grows. A clear version is easy to test. A controlled in-place version can later be introduced where memory or performance requires it.

→ **Example 04 - 01**

Section 3

Function interfaces and explicit data flow

Functions as numerical interfaces

A function is not only a way of avoiding repeated code. In scientific programming, a function is an interface between mathematical ideas and computational objects.

inputs $\longrightarrow$ assumptions $\longrightarrow$ computation $\longrightarrow$ outputs

A good function signature tells us what the computation needs. A good return value tells us what the computation produced. A good implementation checks the assumptions at the boundary, before a numerical mistake propagates into a larger workflow. This is one of the simplest ways to make code more reliable without making it more complicated.

Design from the mathematical map

Start with the mathematical map and then translate it into a function. For example, a Gaussian profile may be viewed as a map

$$(x, x_0, \sigma, A) \mapsto A \exp \left[- \left(\frac{x - x_0}{\sigma} \right)^2 \right].$$

```
1 def gaussian(x, center, width, amplitude=1.0):  
2     z = (x - center) / width  
3     return amplitude * np.exp(-z**2)
```

The function makes the data flow explicit: `x`, `center`, `width`, and `amplitude` enter; an array with the sampled profile leaves.

This is better than constructing the same expression repeatedly in many places, because there is now one clear location where the convention is defined.

Keyword-only arguments for scientific choices

Many scientific parameters have the same type. Accidentally swapping two floating-point arguments may not produce an error, but it can change the experiment.

```
1 def gaussian(x, *, center, width, amplitude=1.0):  
2     z = (x - center) / width  
3     return amplitude * np.exp(-z**2)  
4  
5 x = np.linspace(-5.0, 5.0, 1000)  
6  
7 u = gaussian(x, center=0.0, width=1.5, amplitude=2.0)
```

The symbol `*` makes the following arguments keyword-only. This forces the call site to name the scientific choices.

For small functions used many times, this is often worth the extra characters. It prevents bugs that are syntactically valid but physically wrong.

Separate data from numerical policy

A function often receives two different kinds of information:

- ▶ data that define the object being computed,
- ▶ numerical policy that defines how the computation is performed.

```
1 def derivative_1d(u, x, *, boundary="interior", edge_order=2):
2     if boundary == "interior":
3         return (u[2:] - u[:-2]) / (x[2:] - x[:-2])
4
5     if boundary == "periodic":
6         dx = x[1] - x[0]
7         return (np.roll(u, -1) - np.roll(u, 1)) / (2*dx)
8
9     raise ValueError(f"unknown boundary convention: {boundary}")
```

The data are `u` and `x`. The numerical policy is the boundary convention and edge order. Making this distinction explicit prevents a function from silently implementing one convention while the reader assumes another.

Return-value discipline

A computational function should usually return the quantity it computes. Printing a result, storing it in a global variable, or modifying an object indirectly makes the data flow harder to follow.

```
1 def energy_density(u, ux):  
2     return 0.5*ux**2 + 0.5*u**2  
3  
4 def total_energy(x, u, ux):  
5     density = energy_density(u, ux)  
6     return np.trapezoid(density, x)  
7  
8 E = total_energy(x, u, ux)
```

The result is now a value that can be tested, logged, plotted, compared, or passed to another function.

This discipline is important in notebooks as well. A visible output in a cell is not the same as a reusable return value.

Diagnostics as explicit outputs

Scientific computations often produce more than one meaningful quantity. Instead of recomputing diagnostics in several places, we can collect them in one small function.

```
1 def field_diagnostics(x, u):
2     mass = np.trapezoid(u, x)
3
4     x_cm = np.trapezoid(x*u, x) / mass
5
6     i_peak = np.argmax(u)
7     x_peak = x[i_peak]
8     u_peak = u[i_peak]
9
10    return {
11        "mass": mass,
12        "center": x_cm,
13        "x_peak": x_peak,
14        "u_peak": u_peak,
15    }
```

A dictionary is readable for a small set of named results. The important idea is that diagnostics are returned as data, not only printed as text.

Mutation should appear in the interface

If a function writes into an existing array, the interface should make this visible. One common pattern is an explicit `out` argument.

```
1 def center(u):  
2     return u - u.mean()  
3  
4 def center_out(u, out):  
5     np.subtract(u, u.mean(), out=out)  
6     return out  
7  
8 u_centered = center(u)  
9  
10 work = np.empty_like(u)  
11 center_out(u, work)
```

The first function returns a new array. The second function writes into an array supplied by the caller.

This is not only a performance issue. It is an interface issue: the caller can see which object will be modified.

Validate at the boundary

Validation should happen at the boundary of a function, where assumptions about the input first become important. Shape errors, non-finite values, or inconsistent grid sizes should fail early.

```
1 def require_1d_field(x, u):
2     if x.ndim != 1:
3         raise ValueError("x must be one-dimensional")
4
5     if u.shape != x.shape:
6         raise ValueError("u must have the same shape as x")
7
8     if not np.isfinite(u).all():
9         raise ValueError("u contains NaN or inf")
```

These checks are not a replacement for understanding the mathematics. They are a guard against silent misuse of the function.

In numerical work, failing early is usually much better than producing a beautiful plot of a wrong object.

Validate numerical assumptions

Some assumptions are not only about shape. A derivative may require a sorted grid. A logarithm may require positive data. A normalization may require a non-zero norm.

```
1 def grid_spacing(x):
2     dx = np.diff(x)
3
4     if not np.all(dx > 0.0):
5         raise ValueError("x must be strictly increasing")
6
7     if not np.allclose(dx, dx[0]):
8         raise ValueError("x must be uniformly spaced")
9
10    return dx[0]
```

This kind of validation documents the numerical method. It tells the reader that the code is not a general derivative on an arbitrary grid, but a method written for a specific convention. Explicit restrictions are better than implicit assumptions.

Separate model, method, and diagnostics

A scientific workflow becomes clearer when the model, numerical method, and diagnostics are separate concepts.

```
1 def diffusion_rhs(u, *, dx, diffusion):
2     dudt = np.zeros_like(u)
3     dudt[1:-1] = diffusion * (
4         u[:-2] - 2.0*u[1:-1] + u[2:]
5     ) / dx**2
6     return dudt
7
8 def euler_step(u, *, dt, rhs, rhs_params):
9     return u + dt * rhs(u, **rhs_params)
10
11 def rms_change(u_new, u_old):
12     return np.sqrt(np.mean((u_new - u_old)**2))
```

The model is the right-hand side. The method is the Euler step. The diagnostic is the RMS change.

This separation makes it easier to replace one part of the workflow without rewriting the others.

Passing functions as arguments

Functions are objects in Python. They can be passed to other functions. This is useful when a workflow should be independent of the specific model, residual, or diagnostic being used.

```
1 def mse_loss(model, x, y, params):
2     y_pred = model(x, params)
3     residual = y_pred - y
4     return np.mean(residual**2)
5
6 def linear_model(x, params):
7     return params["a"]*x + params["b"]
8
9 params = {"a": 2.0, "b": -1.0}
10
11 loss = mse_loss(linear_model, x, y, params)
```

This pattern appears in optimization, curve fitting, machine learning, time integration, and Monte Carlo workflows. The outer algorithm does not need to know the internal formula of the model; it only needs a clear interface.

Adapters at library boundaries

Many scientific libraries expect a user-defined function with a specific signature. Instead of hiding parameters globally, write an adapter at the boundary.

```
1 from scipy.integrate import solve_ivp
2
3 def rhs(t, y, *, grid, params):
4     nx = grid["nx"]
5     dx = grid["dx"]
6     D = params["diffusion"]
7
8     u = y.reshape(nx)
9
10    dudt = np.zeros_like(u)
11    dudt[1:-1] = D * (
12        u[:-2] - 2.0*u[1:-1] + u[2:]
13    ) / dx**2
14
15    return dudt.ravel()
```

Our function keeps `grid` and `params` explicit. The solver, however, expects a function of only `t` and `y`.

Solver callbacks as interfaces

The adapter gives the solver the interface it expects while keeping the scientific dependencies visible in our own code.

```
1 sol = solve_ivp(  
2     lambda t, y: rhs(t, y, grid=grid, params=params),  
3     t_span=(0.0, 10.0),  
4     y0=u0.ravel(),  
5 )
```

The `lambda` is not the scientific model. It is an adapter between two interfaces:

- ▶ the solver interface: `f(t, y)`,
- ▶ our interface: `rhs(t, y, grid=..., params=...)`.

This pattern is useful because it avoids hidden globals while still working with libraries that require fixed callback signatures.

→ **Example 04 - 02**

Section 4

Modules, scripts, and lazy data flow

From functions to modules

A function gives a name to one computation. A module gives a name to a group of related computations. In scientific code, this is often the first step from an exploratory notebook to a reusable program.

```
1 # grids.py
2
3 import numpy as np
4
5 def make_grid(xmin, xmax, nx):
6     x = np.linspace(xmin, xmax, nx)
7     dx = x[1] - x[0]
8     return x, dx
```

The module `grids.py` now contains a small, reusable part of the numerical workflow. Other scripts, notebooks, and tests can import it instead of redefining the same grid construction many times.

A module should make useful definitions available. It should usually not run a large simulation just because it was imported.

Modules as namespaces

A namespace is a place where names live. Modules help us avoid having too many unrelated names in one global workspace.

```
1  # diagnostics.py
2
3  import numpy as np
4
5  def rms_error(u, u_ref):
6      residual = u - u_ref
7      return np.sqrt(np.mean(residual**2))
8
9  def max_error(u, u_ref):
10     residual = u - u_ref
11     return np.max(np.abs(residual))
```

The names `rms_error` and `max_error` now belong to the conceptual category “diagnostics”. This is simple, but useful. It makes the code easier to navigate and reduces the need for very long scripts containing everything at once.

A good module answers the question: what kind of tools live here?

Scripts as reproducible experiments

A script should describe one reproducible run: choose parameters, call reusable functions, save or display the result.

```
1  # run_diffusion.py
2
3  from grids import make_grid
4  from models import initial_condition, solve_diffusion
5  from diagnostics import field_diagnostics
6
7  xmin, xmax, nx = -10.0, 10.0, 1001
8  diffusion = 0.1
9  tmax = 5.0
10 x, dx = make_grid(xmin, xmax, nx)
11 u0 = initial_condition(x)
12 u = solve_diffusion(x, u0, diffusion=diffusion, tmax=tmax)
13
14 diag = field_diagnostics(x, u)
15 print(diag)
```

The scientific choices are visible near the top. The computational details are delegated to functions whose interfaces can be inspected separately.

The main pattern

A useful script structure is to put the actual run inside a `main` function and call it only when the file is executed as a script.

```
1 def main():
2     x, dx = make_grid(-10.0, 10.0, 1001)
3     u0 = initial_condition(x)
4     u = solve_diffusion(
5         x, u0,
6         diffusion=0.1,
7         tmax=5.0,
8     )
9     diag = field_diagnostics(x, u)
10    print(diag)
11
12 if __name__ == "__main__":
13     main()
```

This prevents the simulation from running when the file is imported. The same file can now be both executable and safely importable.

A lightweight project layout

A scientific project does not need a complicated software architecture. But even a small amount of organization helps when the code grows beyond one notebook.

```
1 project/  
2     notebooks/  
3         exploration.ipynb  
4     scripts/  
5         run_experiment.py  
6     src/  
7         cernpy/  
8             __init__.py  
9             grids.py  
10            models.py  
11            diagnostics.py  
12            plotting.py  
13     data/  
14     results/
```

The exact layout may vary. The important idea is separation of roles: notebooks for exploration, scripts for reproducible runs, modules for reusable code, data for inputs, and results for generated outputs.

Imports should document dependencies

Imports are part of the readable structure of a program. They tell us which external tools and which internal modules the computation depends on.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from pathlib import Path
5 from scipy.integrate import solve_ivp
6
7 from cernpy.grids import make_grid
8 from cernpy.models import initial_condition
9 from cernpy.diagnostics import field_diagnostics
```

A common convention is to put imports at the top of the file. This makes dependencies visible before the calculation begins.

Wildcard imports such as `from numpy import *` should usually be avoided. They hide where names come from and can accidentally overwrite names from other modules.

Notebooks versus scripts

A notebook is excellent for exploration, visualization, and explanation. A script or module is usually better for code that must be rerun, tested, shared, or used by another workflow. A good compromise is to keep the computational logic in modules and use the notebook as a front end.

```
1 from cernpy.experiment import ExperimentConfig, run_experiment
2 from cernpy.plotting import plot_field
3
4 config = ExperimentConfig(
5     nx=1001,
6     tmax=10.0,
7     diffusion=0.1,
8 )
9
10 result = run_experiment(config)
11
12 plot_field(result.x, result.u)
```

The notebook remains readable, but the numerical logic is not trapped inside hidden notebook state.

Paths and reproducible file locations

Hard-coded absolute paths make code difficult to move between computers, clusters, notebooks, and collaborators. The `pathlib` module gives a clean way to build paths.

```
1 from pathlib import Path
2
3 ROOT = Path(__file__).resolve().parents[1]
4
5 DATA_DIR = ROOT / "data"
6 RESULTS_DIR = ROOT / "results"
7
8 RESULTS_DIR.mkdir(parents=True, exist_ok=True)
9
10 output_file = RESULTS_DIR / "run_001.npz"
11
12 np.savez(output_file, x=x, u=u)
```

This example is appropriate for a script. In a notebook, where `__file__` is not defined, one can set the project root explicitly. The important point is to build paths deliberately instead of scattering strings across the code.

Lazy versus materialized data

A materialized collection stores all elements in memory. A lazy object produces elements only when they are requested.

```
1 # Materialized list: all values are stored  
2 squares_list = [x*x for x in range(10_000)]  
3  
4 # Generator expression: values are produced one by one  
5 squares_lazy = (x*x for x in range(10_000))  
6  
7 total = sum(x*x for x in range(10_000))
```

For small data, the difference may not matter. For large parameter sweeps, large files, or mini-batch workflows, laziness can reduce memory pressure and make the data flow more explicit.

Lazy computation is not automatically faster. Its main advantage is controlled memory use and clearer sequencing.

Generator functions

A generator function uses `yield` to produce values one at a time. This is useful when a workflow should describe a sequence without storing the whole sequence immediately.

```
1 def parameter_sets(diffusions, dampings):
2     for D in diffusions:
3         for gamma in dampings:
4             yield {
5                 "diffusion": D,
6                 "damping": gamma,
7             }
8
9 diffusions = [0.05, 0.1, 0.2]
10 dampings = [0.0, 0.01, 0.05]
11
12 for params in parameter_sets(diffusions, dampings):
13     print(params)
```

This is a lightweight way to represent a parameter sweep. Later, the same generator could feed a solver, a plotting routine, or a batch-processing pipeline.

Mini-batches as a generator

Machine learning and large numerical workflows often process data in batches. A generator makes this interface simple.

```
1 def batches(X, y, batch_size):
2     n = len(X)
3
4     for start in range(0, n, batch_size):
5         stop = min(start + batch_size, n)
6
7         X_batch = X[start:stop]
8         y_batch = y[start:stop]
9
10        yield X_batch, y_batch
11
12
13 for X_batch, y_batch in batches(X_train, y_train, batch_size=64):
14     y_pred = model(X_batch, params)
15     loss = mse_loss(y_pred, y_batch)
```

The training loop does not need to know how batches are produced. It only needs the interface: each iteration gives an input batch and a target batch.

Streaming measurements from a file

Not every dataset must be loaded into memory at once. For line-based text data, a generator can stream measurements one line at a time.

```
1 def read_measurements(path):
2     with open(path, "r", encoding="utf8") as f:
3         for line in f:
4             line = line.strip()
5
6             if not line or line.startswith("#"):
7                 continue
8
9             t, value = line.split()
10
11             yield float(t), float(value)
12
13
14 for t, value in read_measurements("detector_signal.txt"):
15     process_measurement(t, value)
```

This pattern is especially useful for preprocessing, filtering, or extracting diagnostics from large text logs.

Context managers

A context manager controls setup and cleanup. The most common example is opening a file. The file is closed automatically, even if an error occurs inside the block.

```
1 with open("notes.txt", "r", encoding="utf8") as f:
2     first_line = f.readline()
3
4 with np.load("run_001.npz") as data:
5     x = data["x"]
6     u = data["u"]
```

The `with` statement is a tool for controlled state. It marks a region of code where a resource is active: a file is open, a temporary setting is enabled, a timer is running, or a connection exists.

This is cleaner than manually remembering to close or reset things later.

A small timing context manager

Context managers can also be created for scientific bookkeeping. For example, a small timer can mark the runtime of one block without mixing timing code into the computation itself.

```
1 from contextlib import contextmanager
2 from time import perf_counter
3
4 @contextmanager
5 def timer(label):
6     start = perf_counter()
7     try:
8         yield
9     finally:
10         elapsed = perf_counter() - start
11         print(f"{label}: {elapsed:.3f} s")
12
13 with timer("solver"):
14     result = run_experiment(config)
```

This is not a replacement for proper profiling. It is a lightweight tool for making the runtime of important workflow stages visible.

Section 5

Maintainability tools for scientific code

Maintainability is a numerical tool

Maintainability is not separate from scientific correctness. A numerical program is often changed many times: a new parameter is added, a solver is replaced, a boundary condition is modified, a diagnostic is added, or a dataset is scaled differently. A maintainable program makes such changes safer.

visible assumptions + explicit interfaces + controlled state $\longrightarrow$ safer numerical work

This is also a performance issue. Code that is difficult to understand is difficult to profile. Code with unclear interfaces is difficult to accelerate without changing its meaning.

Assertions for internal invariants

An assertion checks something that should always be true if the program is internally consistent.

```
1 def centered_field(u):
2     assert u.ndim == 1
3     assert np.isfinite(u).all()
4
5     return u - u.mean()
6
7
8 def laplacian_1d_interior(u, dx):
9     assert u.ndim == 1
10    assert dx > 0.0
11
12    return (u[:-2] - 2.0*u[1:-1] + u[2:]) / dx**2
```

Assertions are useful during development and testing. They express assumptions close to the code that uses them.

They should not be the only protection against invalid user input, because Python can run with assertions disabled.

Exceptions for invalid inputs

When invalid input can realistically arrive from outside the function, raise an exception with a useful message.

```
1 def make_grid(xmin, xmax, nx):
2     if nx < 2:
3         raise ValueError("nx must be at least 2")
4
5     if xmax <= xmin:
6         raise ValueError("xmax must be larger than xmin")
7
8     x = np.linspace(xmin, xmax, nx)
9     dx = x[1] - x[0]
10
11     return x, dx
```

A good error message should help the user fix the problem. In scientific code, this often means mentioning the violated numerical assumption: shape, positivity, monotonicity, finite values, or compatible units.

Docstrings as scientific contracts

A docstring should not only repeat the function name. It should record the scientific meaning of the inputs and outputs, and any important assumptions.

```
1 def total_energy(x, u, ux):
2     """
3     Compute the discrete total energy of a one-dimensional field.
4
5     Parameters
6     -----
7     x : array, shape (nx,) - Spatial grid.
8     u : array, shape (nx,) - Field values sampled on x.
9     ux : array, shape (nx,) - Approximation of the spatial derivative.
10
11     Returns
12     -----
13     energy : float - Trapezoidal approximation of the total energy.
14     """
15     density = 0.5*ux**2 + 0.5*u**2
16     return np.trapezoid(density, x)
```

A good docstring is part of the interface. It tells future readers what convention the code implements.

Type hints for interfaces

Type hints make interfaces easier to read. They are especially useful when functions are passed around, reused in solvers, or shared between several files.

```
1 import numpy as np
2 from numpy.typing import NDArray
3
4 def rms_error(
5     u: NDArray[np.float64],
6     u_ref: NDArray[np.float64],
7 ) -> float:
8     residual = u - u_ref
9     return float(np.sqrt(np.mean(residual**2)))
```

The annotation says that `u` and `u_ref` are NumPy arrays and that the function returns a scalar float.

Type hints are not a replacement for numerical validation. They usually do not check array shape, units, or finite values at runtime. They are documentation for humans and tools.

Mutable defaults are hidden state

Default argument values are created when the function is defined, not each time it is called. This can create accidental shared state.

```
1  # Problematic: the same list is reused between calls
2  def record_bad(value, history=[]):
3      history.append(value)
4      return history
5
6  print(record_bad(1.0))    # [1.0]
7  print(record_bad(2.0))    # [1.0, 2.0]
8
9
10 # Safer pattern
11 def record(value, history=None):
12     if history is None:
13         history = []
14
15     history.append(value)
16     return history
```

This is a small Python detail, but in scientific workflows it can silently mix diagnostics, parameters, or intermediate results from different runs.

Dataclasses for configuration

A dataclass is a lightweight way to group related values. It is useful when a simulation, model, or preprocessing step depends on several parameters.

```
1 from dataclasses import dataclass
2 from pathlib import Path
3
4 @dataclass(frozen=True)
5 class ExperimentConfig:
6     xmin: float = -10.0
7     xmax: float = 10.0
8     nx: int = 1001
9
10     diffusion: float = 0.1
11     tmax: float = 5.0
12
13     output_dir: Path = Path("results")
```

The configuration is now one named object instead of a loose collection of variables. The option `frozen=True` prevents accidental modification after creation.

This is not heavy object-oriented programming. It is a structured container for scientific choices.

Dataclass validation

A dataclass can validate its own assumptions using `__post_init__`. This keeps parameter checks close to the parameter definition.

```
1 from dataclasses import dataclass
2 @dataclass(frozen=True)
3 class Grid1D:
4     xmin: float
5     xmax: float
6     nx: int
7     def __post_init__(self):
8         if self.nx < 2:
9             raise ValueError("nx must be at least 2")
10        if self.xmax <= self.xmin:
11            raise ValueError("xmax must be larger than xmin")
12    @property
13    def dx(self):
14        return (self.xmax - self.xmin) / (self.nx - 1)
15    def points(self):
16        return np.linspace(self.xmin, self.xmax, self.nx)
```

The object now represents a valid one-dimensional grid specification, not just three unrelated numbers.

Structured results

The same idea can be used for outputs. Instead of returning many unrelated values, we can return a structured result.

```
1 from dataclasses import dataclass
2 import numpy as np
3 from numpy.typing import NDArray
4 @dataclass
5 class ExperimentResult:
6     x: NDArray[np.float64]
7     u: NDArray[np.float64]
8     diagnostics: dict[str, float]
9     result = ExperimentResult(
10         x=x,
11         u=u,
12         diagnostics={
13             "mass": mass,
14             "energy": energy,
15             "max_value": float(np.max(u)),
16         },)
```

This makes later code more readable: `result.x`, `result.u`, and `result.diagnostics` have clear meanings.

One runner function

A useful pattern is to have one function that runs the experiment from a configuration and returns a structured result.

```
1 def run_experiment(config):
2     x = np.linspace(config.xmin, config.xmax, config.nx)
3     u0 = initial_condition(x)
4     u = solve_model(
5         x, u0,
6         diffusion=config.diffusion,
7         tmax=config.tmax,
8     )
9     diagnostics = field_diagnostics(x, u)
10    return ExperimentResult(
11        x=x,
12        u=u,
13        diagnostics=diagnostics,
14    )
```

This function is now a clean interface for scripts, notebooks, tests, and parameter sweeps. The experiment can be repeated without copying the whole workflow.

Replace internals, keep the interface

Good interfaces make performance work safer. If the interface is stable, we can replace the implementation without changing the rest of the project.

```
1 def solve_model(x, u0, diffusion, tmax):  
2     # First implementation: simple NumPy version  
3     return solve_model_numpy(x, u0, diffusion, tmax)  
4  
5  
6 # Later, after profiling:  
7 def solve_model(x, u0, diffusion, tmax):  
8     # Faster implementation with the same interface  
9     return solve_model_numba(x, u0, diffusion, tmax)
```

This is one of the reasons to care about structure before optimization. The goal is not to decorate the code with functions and classes. The goal is to make controlled replacement possible.

Practical checklist

Before turning a notebook or script into a larger scientific workflow, it is useful to ask:

- ▶ Are the important parameters visible?
- ▶ Are function inputs and outputs explicit?
- ▶ Is mutation intentional and clearly named?
- ▶ Can the same computation be rerun from a clean state?
- ▶ Are shape, finite-value, and grid assumptions checked?
- ▶ Are diagnostics returned as values, not only printed?
- ▶ Can a notebook call the computation without containing all of it?
- ▶ Could a slow implementation be replaced without changing the whole project?

Idiomatic scientific Python is not about clever syntax. It is about writing numerical code whose meaning survives debugging, reuse, profiling, and collaboration.

→ **Example 04 - 03**

Section 6

Further reading

Supplementary literature

- ▶ Wilson, G., Aruliah, D.A., Brown, C.T. *et al.* Best Practices for Scientific Computing. *PLOS Biology* **12**(1), e1001745 (2014). DOI: <https://doi.org/10.1371/journal.pbio.1001745>
- ▶ Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., Teal, T.K. Good enough practices in scientific computing. *PLOS Computational Biology* **13**(6), e1005510 (2017). DOI: <https://doi.org/10.1371/journal.pcbi.1005510>
- ▶ Taschuk, M., Wilson, G. Ten simple rules for making research software more robust. *PLOS Computational Biology* **13**(4), e1005412 (2017). DOI: <https://doi.org/10.1371/journal.pcbi.1005412>
- ▶ Rule, A., Birmingham, A., Zuniga, C. *et al.* Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology* **15**(7), e1007007 (2019). DOI: <https://doi.org/10.1371/journal.pcbi.1007007>